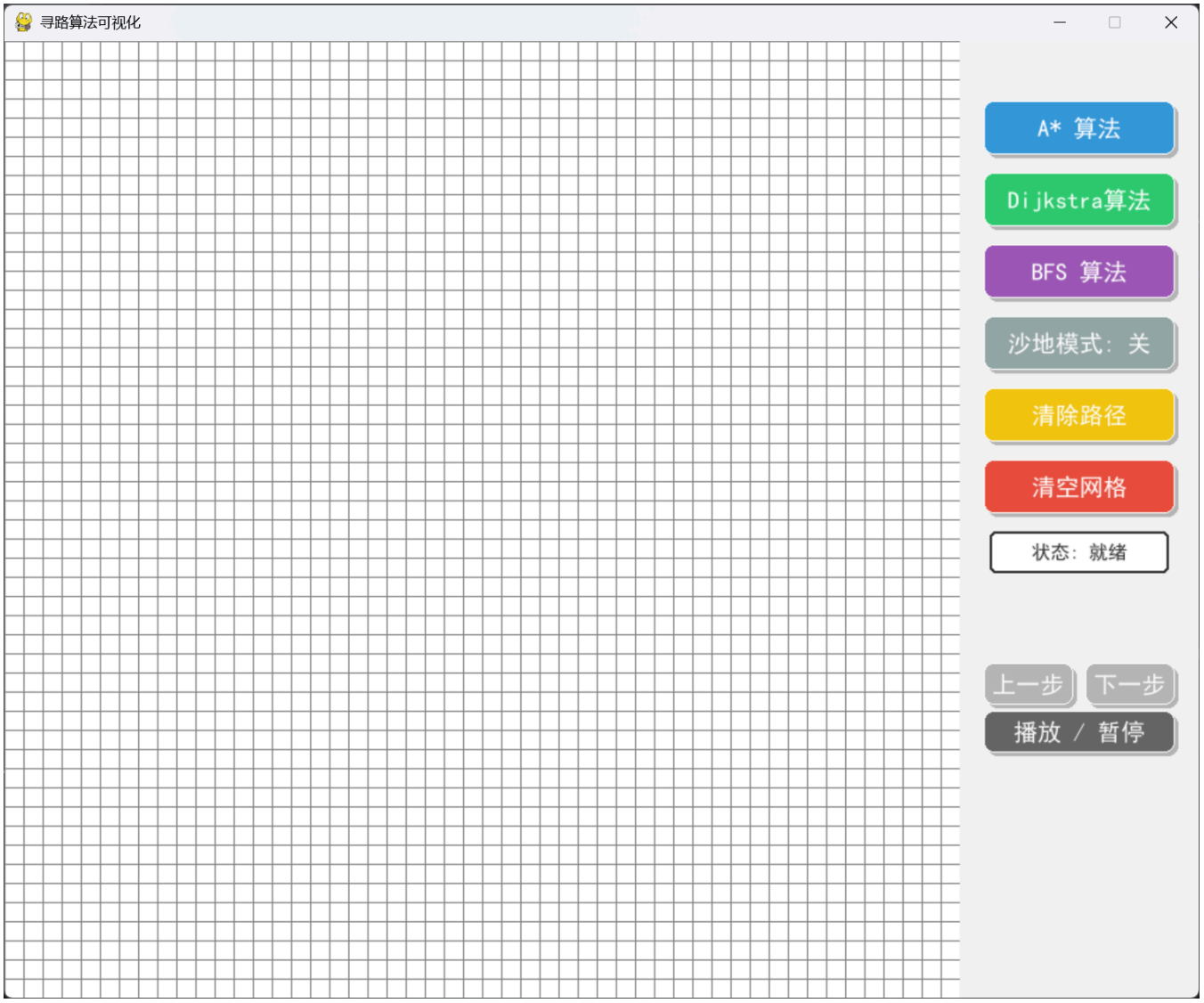
项目报告——寻路算法可视化

1.目标

寻路算法是地图等软件中重要的算法，该项目的目的是在一个网格中，可视化展示出**A*算法**、**Dijkstra算法**和**BFS广度优先算法**三种算法的原理和性能差异，并显示出三个算法具体的探路过程。

2.功能



运行界面如上图所示，用户可以在一个50 * 50的网格中自由布置路径，并提供三种算法的运行视图。

鼠标左键：第一次点击布置起点（橙色），第二次点击布置终点（青色），之后的点击布置障碍（黑色），当沙地模式开启时，之后的点击布置沙地（褐色），支持按住拖动连续布置

鼠标右键：清除网格，支持按住拖动连续清除

沙地模式：关闭时，每个格子的权重默认为1；开启时，可布置带权重的格子（褐色）（简单起见，沙地的权重设置为5）

运行算法：布置好起点和终点后，点击某一算法，会自动运行并给出最短路径（紫色），红色的表示已经探索的格子，绿色的表示准备探索的格子

清除路径：算法运行一次后，点击清除路径按钮，会将探索过程中的染成红色和绿色的格子恢复成**底色**

清空网格：清空全部的网格，恢复成白色

回放：算法执行完成后，可以拖动进度条，进行回溯，查看每一步的结果，以及暂停 / 播放

3.实现说明

代码可视化的思路借鉴了[Red Blob Games](Red Blob Games)中A*算法可视化介绍的演示，项目主要使用Python中的pygame库实现功能。

1）模块化设计

`main.py`：

通过pygame库的调用实现网格界面的设计和显示、算法执行以及鼠标按键交互逻辑等。

`main.py`中的`make_grid`和`draw_grid`负责创建网格与画出网格线，`draw`是核心的绘图函数，通过调用spot类中的draw函数将格子染成不同颜色，在侧边栏画出各个按钮、状态框和进度条等；

`apply_history_state`负责将历史状态快照中的颜色信息还原到网格节点上，`get_clicked_pos`把鼠标点击的像素坐标转换为网格中的行列索引，`handle_grid_click`统一处理网格的点击与拖动逻辑，`reset_path`负责清除探索的路径，对应清除路径按钮；

`run_algorithm`是统一的算法执行入口，三种算法共用一个函数。主体`main`函数是程序主入口，里面定义变量，并监测鼠标事件来执行不同的分支，例如鼠标左键和右键，点击与长按等。

`button.py`：

存放Button类，实现界面中的按钮，因为pygame没有原生的按钮组件，因为要自己定义Button类。

Button类中有文本、颜色和字体等属性，`draw`负责在界面中画出按钮，`is_clicked`判断按钮是否被点击。

`spot.py`：

存放Spot类，将每一个格子定义为一个Spot类，并实现颜色改变和显示逻辑、邻居判断逻辑。

其中的`is_...`函数用于判断格子的是否是某种类型的，`make_...`函数定义了格子的类型；

`reset`用于恢复格子的权重和颜色到初始状态，而`return_to_base_color`恢复格子颜色到底色（当格子染成终点、起点、障碍、沙地时，底色设置为对应的颜色，其他情况时底色仍保持白色），该函数用于清除路径的情况；

`draw`画出格子自身，`update_neighbors`更新格子的邻居：检查上下左右四个方向，如果不是墙，就加入`neighbors`列表。

`pathFinder.py`：

存放寻路算法类，是最重要的部分。

首先定义了`PathFinder`父类，存放三种算法公共的方法：`reconstruct_path`将找到的最短路径显示出来，用于找到最短路径时显示表达；`capture_state`保存当前网格所有格子的颜色快照，用于回溯功能；`find_path`抽象方法，在每个子类中实现自己的`find_path`方法。

BFS广度优先算法

思想：从起点出发，先探索所有距离为 1 的节点，再探索距离为 2 的节点，依此类推，直到找到终点。

核心数据结构：队列（Queue），先进先出，保证按距离起点的步数顺序探索

优点：实现最简单，在等权网格中一定能找到最短路径

缺点：效率较低，而且在非等权网格中，BFS算法找到的结果只是距离最近的路径，而不是权值和最小的路径

时间复杂度：$O(N)$（N 为网格总节点数）

空间复杂度：$O(N)$

Dijkstra算法

思想：从起点出发，始终优先探索到起点实际代价（G 值）最小的节点，直到找到终点。

核心数据结构：优先队列（PriorityQueue），按 G 值排序，G 值越小越先探索

优点：适配非等权图（如网格中沙地代价为5），仍能找到最短路径

缺点：无启发信息，会向所有方向探索，效率比 A* 低

时间复杂度：$O(N \log N)$（N 为网格总节点数），优先队列的插入 / 取出操作时间为 $O(logN)$，最坏需处理所有节点

空间复杂度：$O(N)$

A*算法

思想：启发式优先搜索，结合实际代价（G 值）和预估代价（H 值），优先探索 F 值 = G+H 最小的节点，向终点方向定向探索。在本项目中，H值使用曼哈顿距离衡量

核心数据结构：优先队列，按 F 值排序

优点：有方向性，效率远高于 BFS / Dijkstra算法，在等权 / 非等权图中均能找到最短路径

缺点：实现相对复杂，H 值设计不当会影响效率

时间复杂度：$O(N \log N)$（最坏情况同 Dijkstra），但实际运行中因启发信息，仅需探索少量节点，效率远高于 Dijkstra；

空间复杂度：$O(N)$

`setting.py`：

存放其他文件需要使用的常量，包括颜色RGB常量，状态常量，播放速度等。

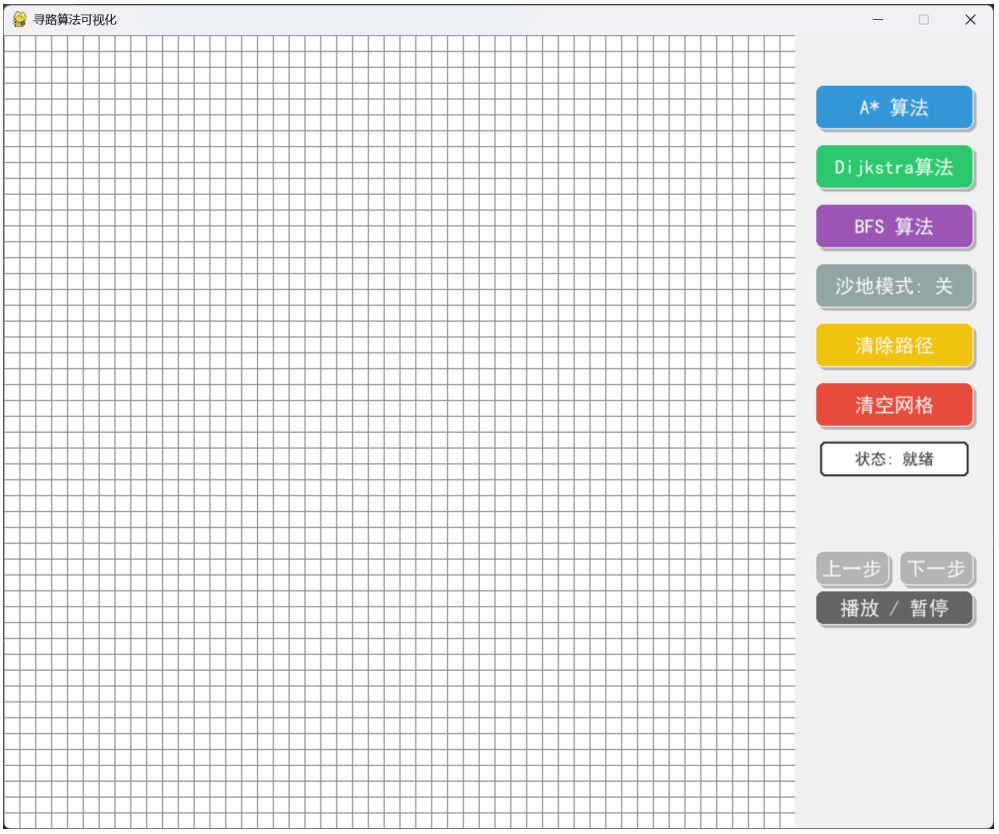
2)版本迭代

项目的第一代主要设计的是A*算法和Dijkstra算法在等权网格中的寻路对比，因为A*算法是目前广泛应用的寻路算法，对比Dijkstra算法有很多优势；

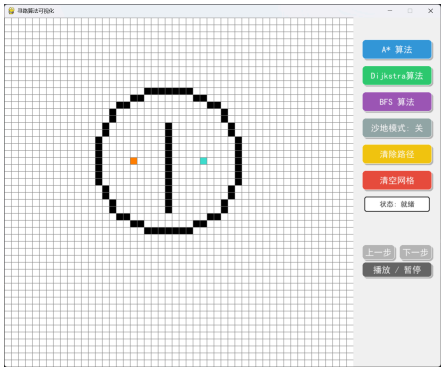
第二代是受到Red Blob Games的灵感启发，增加了寻路过程的回溯功能，方便用户回放算法具体每一步的探路过程；

第三代是我了解到在等权网格中，BFS算法和Dijkstra算法实现的原理是相同的，只是所用的数据结构和性能不同，因此我为代码增加了BFS算法，并增加了非等权模式来展现出Dijkstra算法和BFS算法不一样的情况，说明Dijkstra算法相比于BFS算法的优势。

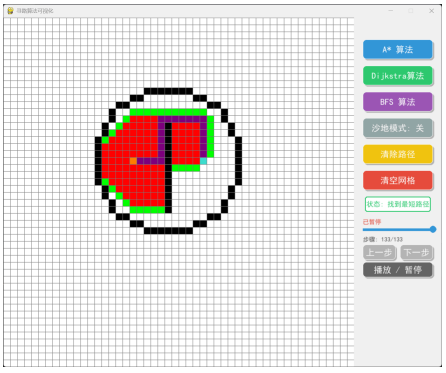
4.结果展示



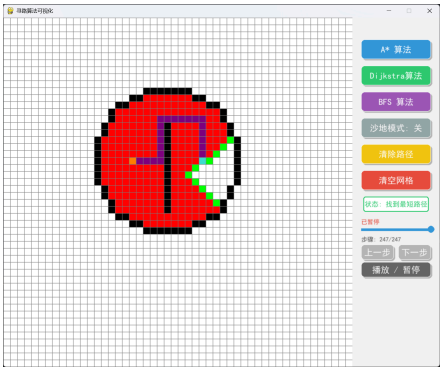
上图是代码运行后弹出的初始窗口，用户可以在左侧网格区域布置起点和终点、障碍和沙地等，如果需要撤销可以使用鼠标右键。右侧是一些功能按钮，点击算法可以在布置好的网格中展示寻路过程，结束后可以进行回放具体的每一步，以及从该步继续执行算法，最后用户可以选择清除路径或清空整个网格。



等权网格场景



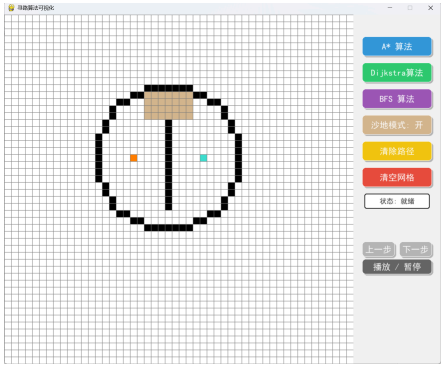
A*算法结果（1）



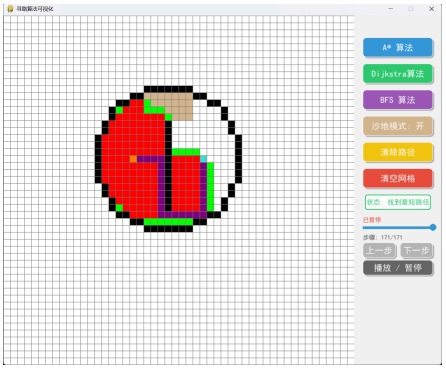
Dijkstra / BFS 算法结果（1）

首先我们布置好了一个等权的网格（没有布置沙地），起点在右边（橙色），目标终点在右边（青色）。

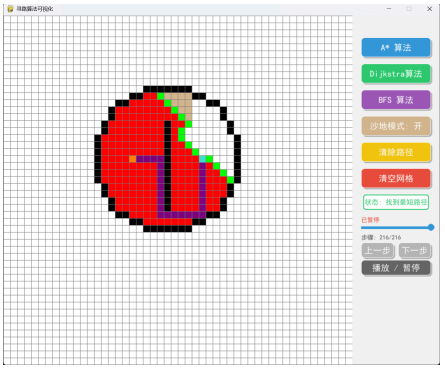
运行A*算法的结果如中间图片所示，运行Dijkstra算法或BFS算法的结果如右图所示。可以看到三种算法的最短路径是相同的，但A*算法明显所需的步骤要少于后两种算法。



非等权网格场景



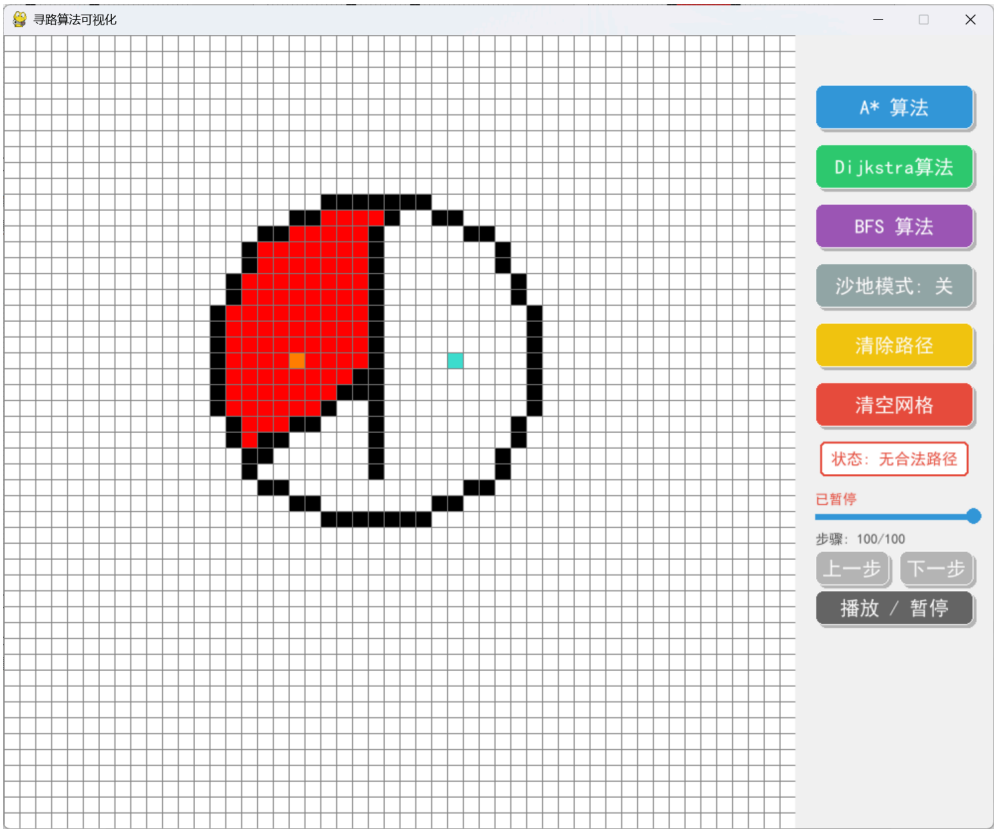
A*算法结果（2）



Dijkstra算法结果（2）

这次我们在上方增加了一块沙地，表示网格的权重为5，重新运行算法，得到图片中的结果。

可以看出A*算法和Dijkstra算法能找到带权的最短路径，但A*算法的性能明显优于Dijkstra算法，步骤更少，而BFS算法找到的结果仍然和等权场景中的相同，不适用于这种情况。



当我们把起点和终点的通路封住，算法无法找到最短路径时，会提示没有合法的路径。